
MEMORIA DESCRIPTIVA

Millán Merino Martínez



GymProgress

19 DE MAYO DE 2026

PROYECTO FINAL
CIFP CAMINO DE LA MIRANDA

Contenido

Introducción	1
Objeto del proyecto	1
Lenguajes empleados.....	2
Despliegue y distribución del sistema.....	3
Requisitos de los clientes	4
Licenciamiento	4
Recursos	5
Hardware:.....	5
Software	6
Humanos	7
Previsión económica del coste del proyecto	7
Descripción de la aplicación	8
Funcionamiento general	8
Arquitectura	10
Interfaz	17
Autoevaluación y conclusiones	19

Introducción

Objeto del proyecto

Mi proyecto tiene como objetivo el diseño, desarrollo e implementación de un sistema informático integral para la gestión, planificación y monitorización del entrenamiento de fuerza y la composición corporal. El proyecto se basa en una arquitectura cliente-servidor compuesta por una API RESTful desarrollada en Java con un framework llamado SpringBoot y una aplicación móvil nativa para el sistema operativo Android.

La motivación principal de este proyecto surge por una mayor concienciación sobre el cuidado físico y la ayuda de los principios científicos para logra este objetivo, tales como la sobrecarga progresiva y el control de la fatiga. Normalmente la mayoría de usuarios de los gimnasios registra de manera ineficiente sus progresos en el gimnasio, utilizando notas de papel, aplicaciones móviles genéricas no adaptadas o directamente no lo registran. Mi proyecto busca solucionar este problema ofreciendo una herramienta centralizada, rápida y

precisa que permite al usuario registrar cada serie, repetición y carga movida de forma intuitiva.

En el mercado existen diversas herramientas comerciales ya consolidadas que abordan este nicho como Striong, Hevy o FitNotes. Aunque estas aplicaciones tienen una gran popularidad y ofrecen un alto grado de madurez, a menudo tienen interfaces sobrecargadas con funcionalidades secundarias, o restringen una parte de sus características como la analítica o el historial con modelos de suscripción económica.

Los aspectos más destacados y novedosos del proyecto radican en su enfoque funcional y técnico:

- ✓ **A nivel funcional:** se da prioridad a que la experiencia del usuario sea ágil y directa. Añadiendo variables avanzadas para controlar la auto regulación como el RPE (Escala de Esfuerzo Percibido) en el registro de series, un catálogo de rutinas globales y estandarizado y terminado con el seguimiento del peso y el porcentaje de grasa del usuario.
- ✓ **A nivel técnico:** el proyecto destaca por su rigor en el diseño de software. Se implementa un modelo de base de datos relacional (MySQL) normalizado con un enfoque moderno basado en Code-First y mapeo objeto-relacional (ORM Hibernate). También garantiza una alta escalabilidad, contraseñas encriptadas para la seguridad de los usuarios y una comunicación cliente-servidor altamente eficiente gracias a la librería Retrofit en el entorno Android.

Lenguajes empleados

Lenguajes de programación y marcado:

- ✓ **Java:** Usado como lenguaje principal para el desarrollo de la lógica del servidor (Backend) usando el framework Spring Boot. He elegido este lenguaje ya que es un lenguaje fuertemente tipado, robusto, orientado a objetos facilitando la implementación de arquitecturas seguras y escalables.
- ✓ **Kotlin:** Utilizado para el desarrollo nativo de la aplicación cliente (Frontend) en el sistema operativo Android. He optado por Kotlin ya que es el lenguaje oficial recomendado por Google, además de su compatibilidad con Java, su sintaxis sencilla y la seguridad que ofrece contra errores de referencia nulas.
- ✓ **SQL:** Usado para la definición y la manipulación de datos en el sistema gestor de base de datos MySQL. Aunque su generación está automatizada gracias a ORM

Hibernate, comprenderlo es necesario para garantizar la integridad y normalización del modelo relacional.

- ✓ **XML y JSON:** Uso XML como lenguaje de marcado para el diseño de las interfaces de usuario y la configuración de manifiesto en Android. JSON se usa para las peticiones y respuestas HTTP entre la aplicación móvil y la API REST.

Entornos de Desarrollo Integrados (IDEs)

- ✓ **IntelliJ IDEA:** Usado para la programación de la API Backend. Ha sido desarrollado por JetBrains y es considerado el líder para Java y Spring Boot. Lo uso por sus altas capacidades de refactorización y depuración en tiempo real.
- ✓ **Android Studio:** Es el IDE oficial de Google usado para la construcción, testeo y empaqueta de la aplicación móvil. Se basa en la misma tecnología que IntelliJ y lo he escogido ya que proporciona herramientas exclusivas como el emulador de dispositivos virtuales, el gestor de dependencias Gradle y un editor visual de interfaces.

Despliegue y distribución del sistema

Ya que el proyecto tiene la arquitectura cliente-servidor, el proceso de despliegue y distribución se divide en dos: el alojamiento de la API (Backend) y la distribución de la aplicación (Frontend).

Despliegue del Backend (API REST y Base de Datos):

La lógica del servidor se compila y ejecuta usando Maven en un archivo ejecutable con formato .jar. Este tipo de empaquetado tiene la ventaja de incluir un servidor web Apache Tomcat embebido simplificando su despliegue.

Para que el servidor funcione necesita un Servidor Virtual Privado (VPS) o una Plataforma como Servicio (PaaS) orientada a la nube. El servidor deberá tener estas características:

- ✓ Disponer de un Entorno de Ejecución de Java.
- ✓ Tener la capacidad de habilitar y exponer puertos de red seguros para recibir y contestar las peticiones REST.
- ✓ Integrar o tener conexión en la misma red privada virtual con un Sistema Gestor de Bases de Datos Relacional (MySQL) para que la información sea persistente.

Distribución de la aplicación cliente (Android)

La distribución del cliente seguirá los estándares del sistema operativo móvil Android. El entorno Android Studio compila el código fuente en un paquete instalable.

Para las fases de prueba, depuración y evaluación del proyecto final la distribución se hará generando un archivo ejecutable APK que puede ser transferido e instalado directamente en cualquier dispositivo físico o emulador.

Para su posible comercialización y distribución el sistema será empaquetado en formato AAB que es el estándar optimizado y requerido por Google Play Store para que la publicación sea oficial, permitiendo que la tienda distribuya únicamente el código y los recursos necesarios.

Requisitos de los clientes

Para los clientes que quiera utilizar la aplicación GymProgress tengan una experiencia de usuario fluida tiene que cumplir con una serie de requisitos técnicos y funcionales:

- ✓ **Dispositivo de Hardware y Sistema Operativo:** el usuario debe usar un dispositivo móvil que posea el sistema operativo Android. Para que las librerías modernas usadas en el desarrollo de la interfaz y la gestión de red (como Retrofit) funcionen, el dispositivo debe contar como mínimo con la versión 8.0 Android.
- ✓ **Conectividad a internet:** como la aplicación sigue el modelo cliente-servidor es indispensable contar con una conexión a internet estable, puede ser a través de redes móviles o redes inalámbricas. Esto es necesario para el envío y recepción de los paquetes de datos JSON de la API RESTful alojada en el servidor para garantizar la persistencia y la sincronización de los entrenamientos.
- ✓ **Autenticación y Credenciales de Acceso:** para la privacidad de la información de salud y rendimiento de cada usuario, el acceso estará restringido. El cliente tiene que completar un registro inicial dando un correo electrónico válido y una contraseña. Luego se necesitarán estas credenciales para iniciar sesión en la aplicación y acceder a su perfil, donde se almacena sus rutinas, medias corporales y registros de series.
- ✓ **Permisos de la Aplicación:** el usuario debe conceder permisos básicos de acceso a la red durante la instalación o la primera vez que se ejecute la aplicación.

Licenciamiento

Todo el software desarrollado en el proyecto estará bajo los términos de la licencia MIT, un modelo de licencia de código abierto. Esta licencia permite el uso, la copia, modificación, fusión, publicación y distribución del software con la condición de que incluya una copia de

la licencia original y el aviso de derechos de autor en todas las copias del código. Gracias al uso de esta licencia se fomenta la divulgación tecnológica en el ámbito académico y facilita posibles ampliaciones del proyecto.

En el proyecto he usado varias librerías, frameworks, y herramientas de terceros respetando los términos y condiciones de estos componentes:

- ✓ **Java y Spring Boot:** El framework utilizado para la API REST del servidor se distribuye bajo la licencia Apache License 2.0, una licencia permisiva que permite que las aplicaciones que lo usen sean de código abierto.
- ✓ **Hibernate ORM:** encargado del mapeo objeto-relacional tiene la licencia GNU Lesser General Public Licenser (LGPL v2.1). Esta licencia permite conectar dinámicamente la Librería de Hibernate en el proyecto.
- ✓ **MySQL:** se usa la versión MySQL community Server bajo la GNU GPLv2. Como es un servicio externo con el que se comunica nuestra API y no tiene código en la aplicación, se cumple con su licencia sin generar conflictos legales.
- ✓ **Android y Kotlin:** tanto Kotlin como el SDK de Android operan bajo la licencia Apache License 2.0.
- ✓ **Retrofit y Gson:** son librerías de terceros desarrolladas por la empresa Square Inc. Que se distribuyen bajo la Apache License 2.0.

Recursos

Hardware:

- ✓ **Para el desarrollo e implementación local:**
 - **CPU:** AMD Ryzen 7 7700X. Su arquitectura multinúcleo y su frecuencia de reloj permiten una compilación muy rápida, reduciendo los tiempos de construcción.
 - **Memoria RAM:** 32GB de memoria principal con una velocidad de 6000 MT/s. Este es uno de los componentes más importantes porque permite la ejecución de IntelliJ IDEA, Android Studio, el servidor Apache, MySQL y la máquina virtual de Android sin latencia.
 - **Almacenamiento:** SSD M.2 con una capacidad de 2TB. Tiene tasas de lectura y escritura ultrarrápida para el manejo de muchos archivos pequeños de los repositorios ya para el arranque instantáneo de los servicios.

- **GPU:** NVIDIA GeForce RTX 4070. Se usa principal mente para la aceleración por hardware para renderizar de manera fluida la interfaz gráfica de los emuladores Android.
- ✓ **Para la Distribución:**
 - **Para el Backend:** la distribución del .jar y la base de datos requiere un Servidor Virtual Privado básico. Un servidor con 1 o 2 núcleos virtuales y 2GB de memoria RAM es suficiente para gestionar las peticiones HTTP.
 - **Para el Frontend:** el hardware requerido depende del cliente, lo único necesario es un dispositivo inteligente con Android 8.0 o superior.

Software

- ✓ **Software de Desarrollo e Implementación (Backend)**
 - **JDK:** He utilizado la versión 25 LTS de java como base para la compilación y ejecución del código del servidor.
 - **Spring Boot:** es el framework principal. Permite la creación de aplicaciones auto configurables y listas para la producción.
 - **Apache Maven:** se encarga de descargar automáticamente las librerías necesarias y compilar el proyecto.
 - **MySQL Community Server:** Sistema Gestor de Bases de Datos Relacionales.
- ✓ **Software de Desarrollo e Implementación (Frontend):**
 - **Android SDK:** es un conjunto de herramientas desarrollo que incluyen librerías de la API de Android, el compilador y un emulador de dispositivos para realizar pruebas.
 - **Gradle:** un sistema de automatización de construcción avanzado que usa Android Studio. Gestión librerías externas y empaqueta los recursos gráficos y el código fuente.
- ✓ **Herramientas Auxiliar y de Pruebas:**
 - **Postman:** la he usado para probar las APIs, simulando peticiones HTTP del cliente, verificar los códigos de estado de las respuestas y la estructura de los JSON devueltos antes de integrarlo en la aplicación móvil.
 - **DBeaver:** una herramienta de administración de bases de datos universal. Usado para ver el esquema relacional, comprobar las tablas generadas por Hibernate y exportar diagramas Entidad-Relación.

- **Git y GitHub:** Git es un sistema de control de versiones y GitHub es una plataforma de alojamiento en la nube, ayudando a mantener un registro de los cambios efectos en el código.
- ✓ **Software para la Distribución:**
 - **Apache Tomcat:** Spring Boot incorpora un servidor Tomcat dentro del .jar permitiendo crear el servicio con una simple línea de comando.
 - **Android OS:** el software requerido del cliente es el sistema operativo Android, que interpreta y ejecuta el código.

Humanos

- ✓ **Fase de Análisis y Diseño (5 horas)**
- ✓ **Fase de Desarrollo del Servidor / Backend (15 horas)**
- ✓ **Fase de Desarrollo Cliente / Frontend (40 horas)**
- ✓ **Fase de Integración y Pruebas / Testing (10 horas)**
- ✓ **Fase de Documentación y Redacción de la memoria(5 horas)**

Previsión económica del coste del proyecto

Para establece el coste total de desarrollo de GymProgress, se ha realizado una estimación realista categorizando los gastos en tres partidas fundamentales: licencias de software, costes de infraestructura para el despliegue y costes de personal humano.

Costes de Software

Gran parte del desarrollo de la aplicación y el servidor se ha usado tecnologías de código abierto (Open Source) o licencias gratuitas para uso educativo y de desarrollo por lo que no supone un gasto económico.

- ✓ **Entornos de desarrollo:** Android Studio (Gratuito) e IntelliJ IDEA (Licencia Community). Coste: 0,00€
- ✓ **Gestión de Bases de Datos:** DBeaver y MySQL Community Server. Coste: 0,00€

Subtotal Software: 0,00€

Costes de infraestructura y Despliegue

Estos costes contemplan los servicios estrictamente necesarios para que el sistema esté operativo, alojado en la nube y disponible para su distribución a los usuarios finales. Se calcula una estimación para el primer año de vida del proyecto.

- ✓ **Servidor VPS (Backend y Base de Datos):** Servidor básico (1-2 núcleos, 2GB RAM) necesario para alojar la API Spring Boot y el sistema gestor MySQL. Estimación de 6 €/mes durante 12 meses. Coste: 72,00 €.
- ✓ **Licencia de Google Play Developer:** Pago único exigido por Google para habilitar la cuenta de desarrollador y poder publicar la aplicación Android en la tienda oficial. Coste: 25,00 €.

Subtotal Infraestructura: 97,00 €

3. Costes de Personal (Recursos Humanos)

Se ha estimado el coste de la mano de obra basándose en el salario medio de un desarrollador de software Junior, fijando un coste por hora de 18 €/h. Las horas invertidas se desglosan según las fases de ciclo de vida del proyecto:

- ✓ Fase de Análisis y Diseño (5 horas): 90,00 €
- ✓ Fase de Desarrollo del Servidor / Backend (15 horas): 270,00 €
- ✓ Fase de Desarrollo Cliente / Frontend (40 horas estimadas): 720,00 €
- ✓ Fase de Integración y Pruebas / Testing (10 horas estimadas): 180,00 €
- ✓ Fase de Documentación y Redacción de la Memoria (5 horas): 90,00 €

Subtotal Personal (75 horas totales): 1.350,00 €

Resumen del Presupuesto Total

Licencias de Software	0,00€
Infraestructura (Primer año)	97,00€
Personal (Desarrollo y Diseño)	1.350,00€
TOTAL	1447,00€

Descripción de la aplicación

Funcionamiento general

Descripción detallada del funcionamiento de la herramienta:

GymProgress es una aplicación móvil diseñada para la ayudar a los usuarios a gestionar su entrenamiento físico. La aplicación permite construir una base de datos centralizada y estandarizada de ejercicios y rutinas predefinidas. A su vez tiene un entorno en tiempo real para que los usuarios puedan guiar sus sesiones de gimnasio con funciones como cronometrar, calcular la capacidad máxima de levantamiento (1RM) y registrando las series, repeticiones, pesos y esfuerzo percibido (RPE). Una vez terminado el entrenamiento la herramienta lo procesa, consolida un historial detallado y genera graficas del progreso físico a lo largo del tiempo.

Enumeración de los Roles

En la aplicación existen dos roles principales:

- ✓ **Usuario (Estándar):** Es el consumidor final de la herramienta. Su rol se centra en la ejecución de entrenamientos, el uso de las calculadoras integradas y el registro de su evolución corporal (peso y porcentaje de grasa).
- ✓ **Administrador:** su nivel de acceso es superior, aparte de poner todas las funciones de un usuario estándar también tiene acceso exclusivo a un panel de control desde el cual puede gestionar, crear y estructurar el contenido global (catálogo de ejercicios y rutinas) que luego usaran los usuarios.

Enumeración de Funcionalidades

Funcionalidades de Administrador:

- ✓ **Añadir Ejercicio Nuevo:** permite añadir un ejercicio al catálogo de ejercicios definiendo el nombre, grupo muscular, descripción y tipo de equipamiento.
- ✓ **Crear Rutina:** Crea un conjunto de ejercicios existentes, definiendo un nombre.

Funcionalidades de Usuario:

- ✓ **Entrenamiento libre:** inicia una sesión de entrenamiento completamente en blanco y va añadiendo ejercicio.
- ✓ **Entrenar Rutina del Catálogo:** selecciona una rutina predefinida que carga los ejercicios que se van a realizar en la sesión.
- ✓ **Registro de series:** función para llevar en tiempo real un registro de las repeticiones, el peso y el nivel de esfuerzo por cada serie, además de añadir notas.

- ✓ Herramientas en Vivo: como el cronómetro de la sesión o de descanso y la calculadora del 1 RM.
- ✓ Historial y Progreso: permite llevar un registro del peso corporal, porcentaje de grasa y visualización gráfica del progreso o consultar las sesiones pasadas por fecha.

Resultados Generados

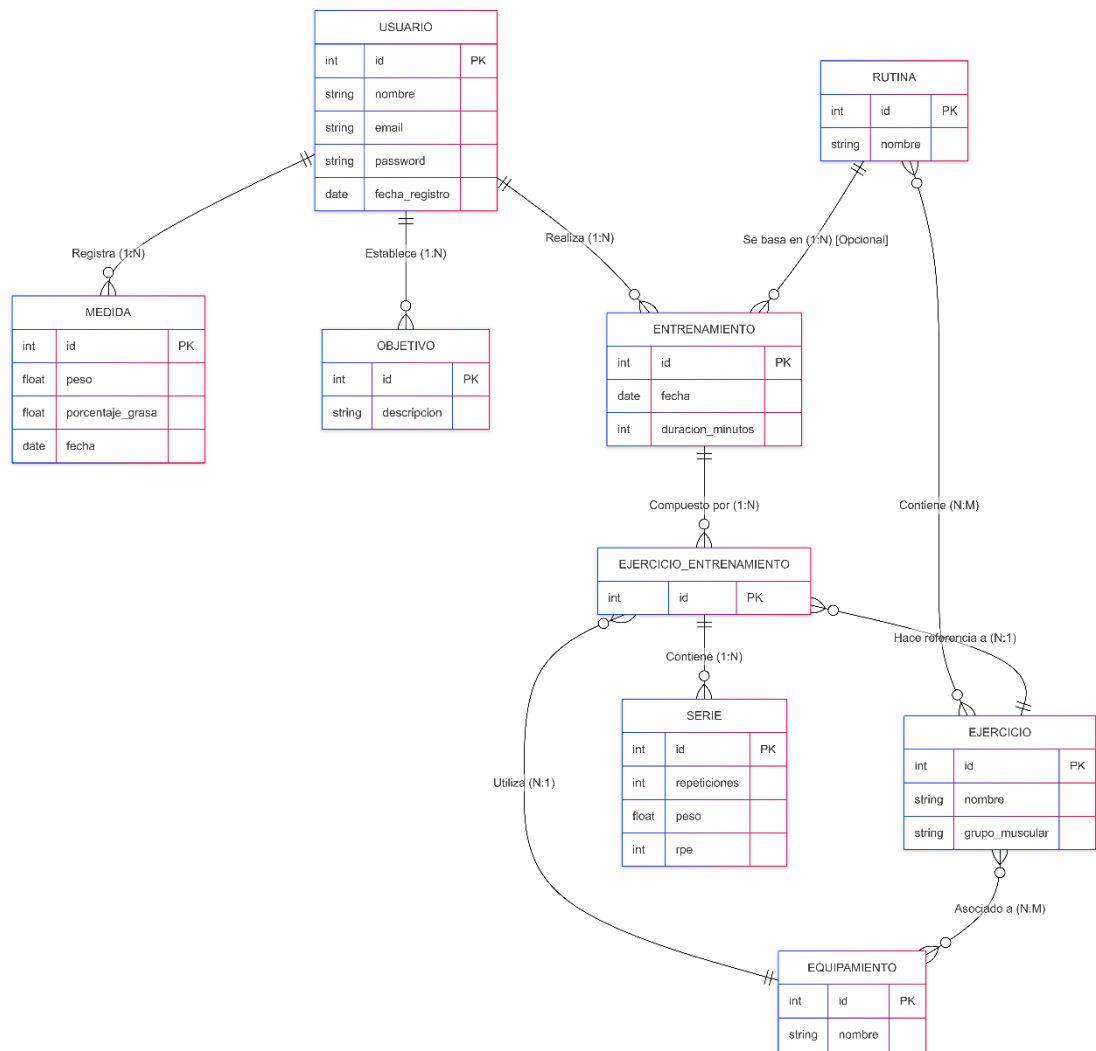
- ✓ Catálogo

Arquitectura

Diseño de la base de datos

Modelo Entidad-Relación

- ✓ Un **Usuario** puede registrar múltiples **Medidas** a lo largo del tiempo (1:N) y puede establecer múltiples **Objetivos** (1:N).
- ✓ Un **Usuario** puede realizar múltiples **Entrenamientos** (1:N).
- ✓ Un **Entrenamiento** puede basarse de forma opcional en un **Rutina** (N:1).
- ✓ Un **Entrenamiento** está compuesto por múltiples bloques de ejercicios realizados en esa sesión, representados por **Ejercicio_Entrenamiento** (1:N).
- ✓ Cada bloque de **Ejercicio_Entrenamiento** contiene a su vez múltiples **Series** (1:N).
- ✓ Cada bloque de **Ejercicio_Entrenamiento** hace referencia a un **Ejercicio** específico del catálogo (N:1) y a un **Equipamiento** específico utilizado en esa sesión (N:1).
- ✓ Las **Rutinas** conforma un catálogo global y contienen múltiples **Ejercicios** (N:M a través de la tabla rutina_ejercicio).
- ✓ Los **Ejercicios** del catálogo global pueden estar asociados a múltiples tipos de **Equipamiento** (N:M a través de la tabla ejercicio_equipamiento).



Modelo Relacional

- ✓ **usuarios** (id_usuario [PK], nombre, email, password, fecha_registro)
- ✓ **medidas** (id_medida [PK], peso_corporal, porcentaje_grasa, fecha, id_usuario [FK])
- ✓ **objetivos** (id [PK], tipo, valor_objetivo, fecha_limite, completado, usuario_id [FK])
- ✓ **equipamiento** (id [PK], nombre)
- ✓ **ejercicios** (id_ejercicio [PK], nombre, descripción, grupo_muscular)
- ✓ **rutinas** (id_rutina [PK], nombre)
- ✓ **rutina_ejercicio** (id_rutina [PK][FK], id_ejercicio [PK][FK])
- ✓ **entrenamientos** (id_entrenamiento [PK], fecha, duracion_minutos, id_usuario [FK], id_rutina [FK])
- ✓ **ejercicio_entrenamiento** (id [PK], orden, notas, ejercicio_id [FK], entrenamiento_id [FK], equipamiento_id [FK])
- ✓ **series** (id_serie [PK], tipo, repeticiones, peso, rpe, ejercicio_entrenamiento_id [FK])

Diccionario de Datos

Usuarios

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Nombre	VARCHAR(255)	NOT NULL	Nombre completo o alias del usuario
Email	VARCHAR(255)	NOT NULL	Correo electrónico (credencial de acceso)
Password	VARCHAR(255)	NOT NULL	Contraseña encriptada por seguridad
Fecha_registro	DATE TIME	NOT NULL	Fecha y hora en la que se creó el registro

Medidas

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de medidas
Peso	DOUBLE	NOT NULL	Peso del usuario en kilogramos
Porcentaje_grasa	DOUBLE	Opcinal	Índice de grasa corporal del usuario
Fecha	DATE TIME	NOT NULL	Fecha exacta a de la toma de medidas
Usuario_id	BIGINT	FK	Referencia de usuario al que pertenece la medida

Objetivos

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto_increment	Identificador único de objetivo
Tipo	VARCHAR(255)	NOT NULL	Tipo de objetivo (ej, Peso, Grasa, Fuerza...)
valor_objetivo	DOUBLE	NOT NULL	Valor numérico a alcanzar
Fecha_limite	DATE	NULL	Fecha límite estimada para cumplir el objetivo
Completado	BIT(1)	NOT NULL	Indica si el objetivo se ha cumplido (0 o 1)
Usuario_id	BIGINT	FK, NULL	Referencia al usuario (usuarios.id)

Equipamiento

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de equipamiento
Nombre	VARCHAR(255)	UNIQUE, NOT NULL	Nombre del equipamiento

Ejercicios

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Nombre	VARCHAR(255)	NOT NULL	Nombre del movimiento

Grupo_muscular	VARCHAR(255)	NOT NULL	Zona anatómica principal implicada
----------------	--------------	----------	------------------------------------

Ejercicio_equipamiento

Atributo	Tipo de dato SQL	Restricción	Descripción
Ejercicio_id	BIGINT	PK*, FK, NOT NULL	Referencia al ejercicio (ejercicios.id)
Equipamiento_id	BIGINT	PK*,FK, NOT NULL	Referencia al equipo compatible (equipamiento.id)

Rutinas

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Nombre	VARCHAR(255)	NOT NULL	Nombre descriptivo

Rutina_ejercicio

Atributo	Tipo de dato SQL	Restricción	Descripción
Rutina_id	BIGINT	PK,FK	Referencia a la tabla rutinas
Ejercicio_id	BIGINT	PK, FK	Referencia a la tabla ejercicios

Entrenamientos

Atributo	Tipo de dato SQL	Restricción	Descripción
----------	------------------	-------------	-------------

Id	BIGINT	PK, Auto-incremental	Identificador único de usuario
Fecha	DATETIME	NOT NULL	Fecha y hora de realización
Duración_minutos	INT	NOT NULL	Tiempo total de duración de la sesión
Usuario_id	BIGINT	FK	Referencia al usuario que entrenó
Rutina_id	BIGINT	FK, Opcional	Referencia a la rutina base seguida (si la hay)

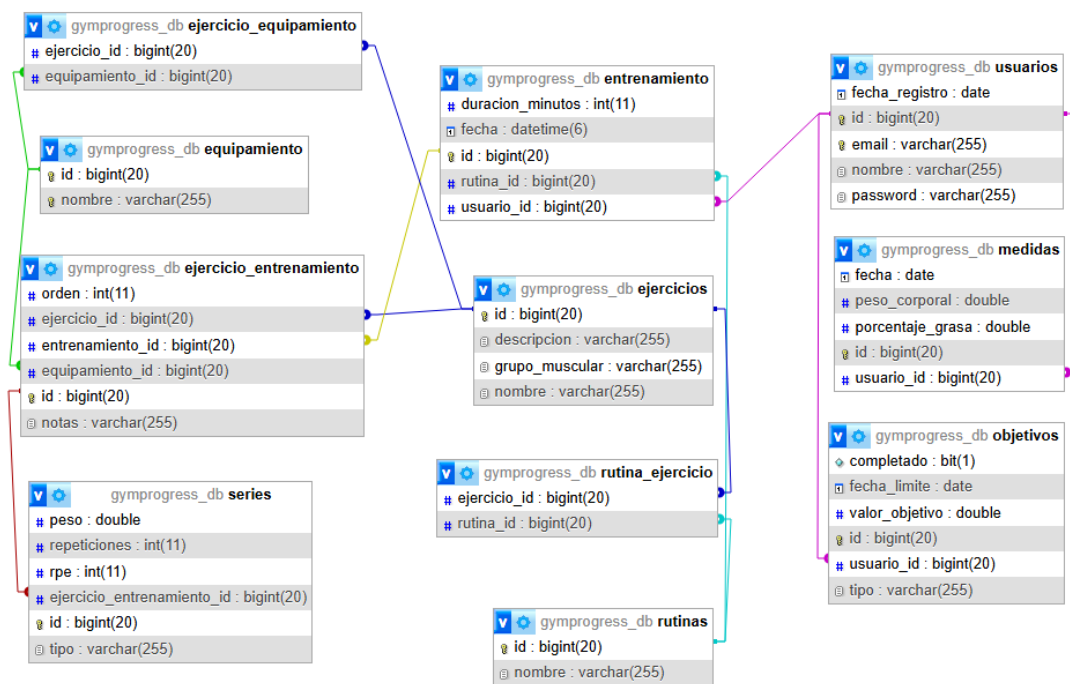
Ejercicio_entrenamiento

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incremental	Identificador del bloque de ejercicio en la sesión.
Orden	INT(11)	NULL	Orden de la ejecución dentro de la sesión
Notas	VARCHAR(255)	NULL	Observaciones específicas para esta ejecución
Ejercicio_id	BIGINT	FK, NLL	Referencia al entrenamiento (ejercicios.id)
Entrenmaiento_id	BIGINT	FK, NULL	Referencia al entrenamiento (entrenamiento.id)
Equipamiento_id	BIGINT	FK, NULL	Equipo particular usado en la sesión (equipamiento.id)

Series

Atributo	Tipo de dato SQL	Restricción	Descripción
Id	BIGINT	PK, Auto-incrementa l	Identificador único de la serie realizada
Tipo	VARCHAR(255)	NULL	Tipo de serie (ej. Efectiva, Calentamiento)
Repeticiones	INT	NOT NULL	Número de alzamientos completados
Peso	DOUBLE	NOT NULL	Carga externa levantada en kilogramos
RPE	INT	NOT NULL	Escla de esfuerzo percibido (1-10)
Ejercicio_entrenamiento_id	BIGINT	FK, NULL	Vincula la serie con el bloque del ejercicio (ejercicio_entrenamiento.id)

Diagrama de clases



Arquitectura del sistema

Interfaz

Características principales

Descripción de la interfaz y diseño general

La interfaz de la realizado priorizando el diseño minimalista, la usabilidad y la accesibilidad. Como la herramienta esta enfocada para usarse durante entrenamientos en el gimnasio, este diseño evita la sobrecarga de información y utiliza una paleta de colores no agresiva.

Visualmente, el diseño se apoya fuertemente en las guías de Material Design, haciendo un gran uso de componentes como “MaterialCardView” para agrupar visualmente la información (por ejemplo, en las listas de ejercicios o en el historial de progreso). Una de las características clave es su adaptabilidad: la aplicación detecta automáticamente el tema del dispositivo del usuario y activa automáticamente el modo claro u oscuro, así garantiza siempre un contraste óptimo y una lectura cómoda de métricas importantes como los pesos y las repeticiones.

Navegación

La navegación de la aplicación es directa y jerárquica, diseñada para que el usuario llegue donde necesita con los mínimos toques posibles.

- ✓ Todo empieza en una pantalla de Login/Registro clara y sin distracciones.
- ✓ Una vez autenticado, el usuario llega a la pantalla principal que actúa como centro de operaciones.
- ✓ Desde ahí, con botones amplios y accesibles (destacados en azul para las acciones principales y rojo para la salida), se ramifica en las distintas funcionalidades:

entrenamientos, rutinas, medidas. La acción para ir atrás sigue el estándar nativo de Android, haciendo que el flujo sea completamente intuitivo.

Lenguajes empleados y validación

Todo el frontend de la aplicación se ha desarrollado de forma nativa para Android.

- ✓ **Lenguajes:** Para el diseño de las vistas (layouts) se ha utilizado XML a través de Android Studio, mientras que la lógica de la interfaz y la interacción con el usuario está programada con Kotlin.
- ✓ **Validación:** Para asegurar la integridad de los datos antes de ser enviados al servidor, la interfaz cuenta con validaciones en los propios formularios. Se comprueban que los campos obligatorios no estén vacíos, se valida el formato del email, y en los entrenamientos se comprueban que los inputs de las métricas contengan valores numéricos coherentes para evitar errores en la base de datos o en las gráficas de progreso.

Usabilidad/accesibilidad

Para el desarrollo de **GymProgress**, se ha tenido muy en cuenta que el entorno de uso principal es una sala de musculación. Esto significa que el usuario va a interactuar con la aplicación entre series, a menudo con fatiga física o prisas, por lo que la interfaz debe ser a prueba de frustraciones.

Mejoras de Usabilidad:

- ✓ **Diseño "Fatigue-Proof" (A prueba de fatiga):** Se han utilizado áreas de toque amplias, cumpliendo con el estándar mínimo de 48dp de Material Design. Botones críticos como "AÑADIR SERIE" o "TERMINAR Y GUARDAR" ocupan todo el ancho de la pantalla para que sea casi imposible fallar al pulsarlos, incluso sin mirar demasiado.
- ✓ **Teclados contextuales:** Para agilizar el registro de datos, los inputs están configurados para desplegar el teclado más eficiente en cada caso. Por ejemplo, al introducir los KG o las repeticiones se abre directamente el teclado numérico (`inputType="numberDecimal"`), evitando que el usuario tenga que cambiar manualmente desde el teclado alfanumérico.
- ✓ **Prevención de errores y feedback visual:** La aplicación bloquea acciones ilógicas para mantener la base de datos limpia. Por ejemplo, el campo RPE está restringido a valores del 1 al 10. Además, se utilizan componentes nativos como *Toasts* para avisar al usuario de que un entrenamiento se ha guardado correctamente o si le falta rellenar algún campo.

Características de Accesibilidad:

- ✓ **Contraste y Tema Dinámico:** Como se comentó en el apartado de diseño general, la app respeta el tema del sistema (Claro/Oscuro). El modo oscuro no solo ahorra batería en pantallas OLED, sino que reduce drásticamente la fatiga visual y evita deslumbramientos en entornos cerrados, mejorando la accesibilidad para usuarios con fotofobia o sensibilidad ocular.
- ✓ **Textos escalables:** Todo el texto de las vistas XML está definido utilizando la unidad de medida sp (Scale-independent Pixels). Esto garantiza que, si un usuario tiene problemas de visión y configura la letra de su sistema operativo más grande, la fuente de GymProgress crecerá en proporción sin romper el *layout*.
- ✓ **Soporte para lectores de pantalla (TalkBack):** Se ha cuidado la semántica de la interfaz añadiendo el atributo `contentDescription` a los elementos visuales que no son texto puro (como el icono del "ojo" para mostrar/ocultar la contraseña en el login o los gráficos del progreso). De esta forma, los usuarios con discapacidad visual que utilicen TalkBack pueden navegar por la app enterándose perfectamente de qué hace cada botón.

Autoevaluación y conclusiones

Valoración del trabajo y dificultades encontradas

A nivel general, el desarrollo de GymProgress ha sido un reto bastante exigente, sobre todo por la decisión de abarcar el proyecto completo como *Full-Stack*. No se trataba solo de hacer unas pantallas bonitas, sino de que todo funcionara de verdad contra un servidor.

Las principales dificultades que me he encontrado han sido:

- ✓ **La curva de aprendizaje y la integración:** Pasar de hacer cosas locales en el móvil a estructurar una API REST robusta. Pelearme con la conexión entre el frontend en Android (Kotlin) y el backend en Spring Boot ha llevado su tiempo, especialmente manejando las peticiones asíncronas y parseando correctamente las respuestas JSON.
- ✓ **Seguridad y autenticación:** Implementar el sistema de seguridad mediante tokens JWT (JSON Web Tokens) me dio bastantes quebraderos de cabeza al principio. Asegurar las rutas, gestionar la expiración del token y mantener la sesión del usuario en la app de Android sin comprometer la seguridad fue uno de los puntos más críticos.

Valoración de la herramienta o aplicación desarrollada

La aplicación cumple con el objetivo principal: registrar entrenamientos de fuerza de forma rápida y sin distracciones. La interfaz limpia basada en Material Design, la implementación del modo oscuro automático y el sistema de gráficas para visualizar la evolución del peso le dan un aspecto muy profesional. Además, contar con un Panel de Administrador deja la puerta abierta a gestionar el catálogo de ejercicios de forma centralizada, lo cual es un punto a favor muy grande de cara a su mantenimiento.

Conclusión final

- ✓ **Del diseño:** He aprendido que "menos es más". Al principio tiendes a querer meter mucha información en pantalla, pero centrarse en la usabilidad y usar componentes estándar (como los MaterialCardView) hace que la app sea mucho más intuitiva para el usuario final.
- ✓ **De la aplicación:** He conseguido montar una arquitectura sólida. Separar bien la lógica de presentación en Kotlin de la lógica de negocio y persistencia en el servidor Spring Boot + MySQL hace que la app sea muy escalable de cara al futuro.
- ✓ **Del tiempo empleado:** Como suele pasar en el desarrollo de software, la planificación inicial se queda corta. Se invierte mucho más tiempo del previsto en depurar errores, cuadrar los *endpoints* de la API y ajustar pequeños detalles de la interfaz para que se vea perfecta en diferentes tamaños de pantalla.
- ✓ **Conocimientos adquiridos:** Ha sido un salto cualitativo brutal. He pasado de ver tecnologías aisladas a entender el ciclo de vida completo de un producto de software: desde el diseño de la base de datos y la creación de la API con Java, hasta el consumo de esos datos y la creación de la interfaz nativa en Android.